

=====
 What is Build & Deployment
 =====

=> Take latest code from Git Hub Repo.

=> Build Source code using Maven

=> Perform Code Review Using Sonar

=> Upload Project Artifact into Nexus

=> Create Docker Image for the application

=> Deploy app in k8s cluster.

=> In single day multiple times code will be committed to git hub repository from Development team so multiple times we have to perform project build and deployment process.

Note: If we do build and deployment process manually then it is time taking process and error prone.

=> To overcome above problems, we need to automate Project Build and Deployment process.

=> To automate project build and deployment process we will use JENKINS.

=====
 JENKINS
 =====

=> Open source Software & free of cost

=> Developed by using Java Language. To run jenkins server we need to install java s/w first.

=> It is called as CI CD Server.

CI : Continuous Integration

CD : Continuos Delivery

=> CI CD is one appraoch to automate project Build & Deployment process.

=> Using Jenkins we can deploy any type of project
 (ex: java, python, dot net, react, angular).

=====
 Jenkins Setup
 =====

@@ Git Repo : <https://github.com/ashokitschool/DevOps-Documents/blob/main/01-Jenkins-Server-Setup.md>

=====
 what is job in jenkins ?
 =====

=> JOB means set of steps that we are giving to jenkins to perform the task

=> In Jenkins JOB we will configure Steps like below

Step-1 : Take code from git hub repo

Step-2 : Perform maven build process

Step-3 : Create Docker Image

Step-4 : push Docker Image to Docker hub

Step-5 : Deploy Docker Image in K8S Cluster

=> Jenkins JOBS we can create in 2 ways

1) Free Style Project (GUI)

2) Pipeline (code)

=====

Creating First Job in Jenkins

1) Goto Jenkins Dashboard

2) Click on New Item

- > Enter Item Name (Job Name)
- > Select Free Style Project & Click OK
- > Enter some description
- > Click on 'Build' tab
- > Click on 'Add Build Step' and select 'Execute Shell'

3) Enter below shellscript

```
echo "Hello Guys,"
touch ashokit.txt
echo "Hello Guys, Welcome to Jenkins Classes" >> ashokit.txt
echo "Done..!!"
```

4) Apply and Save

Note: With above steps we have created JENKINS Job

5) Click on 'Build Now' to start Job execution

6) Click on 'Build Number' and then click on 'Console Output' to see job execution details.

=> Jenkins Home Directory in EC2 : /var/lib/jenkins/workspace/

```
$ cd /var/lib/jenkins/workspace/
```

7) Go to Jenkins home directory and check for the job name --> check the file created inside the job.

=====

Jenkins Job with with GIT Hub Repo + Maven - Integration

Step-1 :: Configure Maven as global tool

(Jenkins Dashboard -> Manage Jenkins --> Tools -> Add maven)

Step-2 :: Take project git repo

Ex: <https://github.com/ashokitschool/maven-web-app.git>

Step-3 :: Create Jenkins job

- > New Item
- > Enter Item Name (Job Name)
- > Select 'Free Style Project' & Click OK
- > Enter some description
- > Go to "Source Code Management" Tab and Select "Git"

- > Enter Project "Git Repo URL"
- > Go to "Build tab"
- > Click on Add Build Step and Select 'Invoke Top Level Maven Targets'
- > Select Maven and enter goals 'clean package'
- > Click on Apply and Save

Note: With above steps we have created JENKINS Job

Step-4 : Click on 'Build Now' to start Job execution.

Step-6 : Click on 'Build Number' and then click on 'Console Output' to see job execution details.

=> Jenkins Home Directory in EC2 : /var/lib/jenkins/workspace/

=> Go to jenkins workspace and then go to job folder then go to target folder there we see war file created.

=====
Jenkins - Master & Slave Architecture
=====

=> In one project multiple microservices will be available and for every microservice one jenkins job is required to automate build and deployment.

=> If we use single machine for jenkins jobs executions, then burden will be increased on that machine and that machine can crash also.

=> To reduce burden on jenkins server we will use Master & Slave Configuration.

=> Master & Slave configuration is used to reduce burden on Jenkins Server by distributing tasks/load.

=====
Jenkins Master
=====

=> The machine which contains Jenkins s/w is called as Jenkins Master machine.

=> It is used to create the jobs

=> It is used to schedule the jobs

=> It is responsible to distribute Jobs execution to slave machines.

Note: We can run jobs on Jenkins Master machine directly but not recommended.

=====
Jenkins Slave
=====

=> The machine which is connected with 'Jenkins Master' machine is called as 'Jenkins-Slave' machine.

=> Slave Machine will receive task from 'Master Machine' for job execution.

=====
Step-1 : Create Jenkins Master vm
=====

- 1) Launch Linux VM (t2.medium)
- 2) Install Java s/w
- 3) Install Jenkins s/w

=====

Step-2 : Create Jenkins Slave vm

=====

- 1) Create EC2 instance (Ubuntu with t2.micro)
- 2) Connect to EC2 using ssh client
- 3) Change hostname for readability

```
$ sudo hostname jenkins-slave
$ exit
```

- 3) Install Java Software

```
$ sudo apt install default-jre
```

- 4) Create one directory in /home/ubuntu

```
$ mkdir slavenode
```

=====

Step-3: Configure Slave Node in Jenkins Master Node

=====

- 1) Go to Jenkins Dashboard
- 2) Go to Manage Jenkins
- 3) Select Nodes option
- 4) Click on 'New Node' -> Enter Node Name -> Select Permanent Agent
- 5) Enter Remote Root Directory

Ex: /home/ubuntu/slavenode

- 6) Enter Label name as Slave-1
- 7) Select Launch Method as 'Launch Agent Via SSH'
- 8) Give Host as 'Slave VM public DNS URL'
- 9) Add Credentials (Select Kind as : SSH Username with private key)
- 10) Enter Username as : ubuntu
- 11) Select Private Key as Enter Directly and add private key

Note: Open pem file and copy content and paste it

- 12) Select Host Key Strategy as 'Manually Trusted Key Verification Strategy'
- 13) Click on Apply and Save (We can see configured slave)

***** With above steps Master and Slave Configuration Completed *****

-> Go to Jenkins Server and Create Jenkins Job

Note: Under Generation Section of Job creation process, Select "Restrict Where This Project Can Run" and enter Slave Node Label name and finish job creation.

-> Execute the Job using 'Build Now' option

Note: Job will be executed on Slave Node (Go to Job Console Output and verify execution details).

=====

Jenkins Pipeline

=====

=> Jenkins Pipeline is a way to define CI CD process as a code.

=> When we are dealing with complex CI CD process then pipelines are highly recommended.

=> Jenkins Pipeline contains set of stages

Stage-1 : Clone git repo

Stage-2 : Maven Build

Stage-3 : Code Review

Stage-4 : Artifact Upload

Stage-5 : Docker Image

Stage-6 : Push Image

Stage-7 : K8S Deployment

Stage-8 : Email Notification

=> We can develop jenkins pipeline in 2 ways

1) Declarative Pipeline

2) Scripted Pipeline (Groovy)

=====

Jenkins Declarative Pipeline Syntax

=====

```
pipeline {
    agent any
    stages {
        stage('git clone'){
            steps{
                // logic
            }
        }
        stage('mvn build'){
            steps{
                // logic
            }
        }
    }
}
```

=====

Jenkins Pipeline with Git + Maven

=====

```
pipeline {
    agent any

    tools {
        maven "Maven-3.9.9"
    }
}
```

```

    }

    stages {
        stage('git clone') {
            steps {
                git 'https://github.com/ashokitschool/maven-web-app.git'
            }
        }
        stage('mvn build'){
            steps{
                sh 'mvn clean package'
            }
        }
    }
}

```

=====

Email Notifications In Jenkins

=====

=> We can configure Email Notifications in Jenkins to inform team members regarding jenkins jobs execution detils.

=> After Jenkins job execution we can trigger email to project team members regarding job execution status.

=> To send emails from Jenkins we need to configure SMTP properties.

SMTP : Simple mail transfer protocol

Note: In realtime we can use company provided SMTP properties but here we can use gmail SMTP properties for practice.

SMTP Server : smtp.gmail.com

SMTP Port : 587

Username : <your-gmail>

Password : <your-gmail-app-pwd> (not gmail login pwd)

@@@ URL To generate gmail app pwd : <https://g.co/kgs/P754MXf>

=====

How to configure SMTP props in jenkins

=====

=> Go to manage jenkins

=> Go to system

=> Go to "Extended Email Notification" section

=> Add SMTP details like server and port

=> Add authentication with gmail id and gmail-app-pwd

=> Select TLS checkbox for email encryption

Note: In jenkins pipeline we can add below post section to send email using "emailext" plugin.

```

-----

post{

```

```

failure {
    emailx(
        subject : "Build Failed ${env.JOB_NAME} #${env.BUILD_NUMBER}",
        body: "The build got failed",
        to : "",
        from: "",
        attachLog: true
    )
}

success {
    emailx(
        subject : "Build Success ${env.JOB_NAME} #${env.BUILD_NUMBER}",
        body: "The build got success",
        to : "abcteam@oracle.com",
        from: "",
        attachLog: true
    )
}
}

```

```

pipeline {
    agent any

    tools {
        maven "Maven-3.9.9"
    }

    stages {
        stage('git clone') {
            steps {
                git 'https://github.com/ashokitschool/maven-web-app123.git'
            }
        }
        stage('mvn build'){
            steps{
                sh 'mvn clean package'
            }
        }
    }

    post{
        failure {
            emailx(
                subject: "Build Failed ${env.JOB_NAME} #${env.BUILD_NUMBER}",
                body: "The build got failed",
                to: "ashokitschool@gmail.com",
                from: "ashokit.classes@gmail.com",
                attachLog: true
            )
        }

        success {
            emailx(
                subject : "Build Success",
                body: "The build got success",
                to : "ashokitschool@gmail.com",
                from: "ashokit.classes@gmail.com",
                attachLog: true
            )
        }
    }
}

```

```
    }
}
}
```

How to work with parallel stages in Jenkins pipeline

=> Jenkins job contains multiple stages like below and all the stages will be executed sequentially.

```
pipeline {
    agent any

    stages {
        stage('git clone'){
            steps{
                echo 'git cloning.....'
            }
        }
        stage('mvn build'){
            steps{
                echo 'maven build.....'
            }
        }
        stage('code review'){
            steps{
                echo 'code review.....'
            }
        }
        stage('nexus upload'){
            steps{
                echo 'nexus upload'
            }
        }
        stage('docker image'){
            steps{
                echo 'docker image creation'
            }
        }
    }
}
```

=> Inorder to save time, we can execute multiple stages parallelly like below

```
pipeline {
    agent any

    stages {
        stage('git clone'){
            steps{
                echo 'git cloning.....'
            }
        }
        stage('mvn build'){
            steps{
                echo 'maven build.....'
            }
        }
    }
}
```



```

    }

    stage('Paralell Stages'){
        parallel{
            stage('code review'){
                steps{
                    echo 'code review.....'
                }
            }
            stage('nexus upload'){
                steps{
                    echo 'nexus upload'
                }
            }
        }
    }
    stage('docker image'){
        steps{
            echo 'docker image creation'
        }
    }
}
}
}

```

```

=====
Jenkins Scripted Pipelines
=====

```

=> Scripted pipelines are used to define CI CD workflow using groovy scripting.

=> Scripted pipeline will provide more flexibility and more control on pipeline stages execution.

=> We can implement error handling logic in programmatic way using scripted pipelines.

----- Scripted Pipeline syntax -----

```

node {

    stage('git clone'){
        echo 'cloning..'
    }

    stage('mvn build'){
        echo 'mvn build in progress'
    }
}

```

-----pipeline with error handling-----

```

node {

    try{
        stage('git clone'){
            echo 'cloning..'
        }

        stage('mvn build'){
            echo 'mvn build in progress'
        }
    }catch(err){
        echo "â€œ ERROR: ${err.getMessage()}"
        // logic to send email
    }
}

```

```
}
```

```
=====
What is hybrid pipeline ?
=====
```

=> A Hybrid Jenkins Pipeline means we can combine both declarative and scripted syntaxes in single pipeline.

```
pipeline {
    agent any

    stages {
        stage('git clone'){
            steps{
                git <repo-url>
            }
        }
        stage('mvn build'){
            steps{
                script{
                    // groovy scripting
                }
            }
        }
    }
}
```

```
=====
User Management
=====
```

=> In our project multiple teams will be available.

- 1) Development Team
- 2) Testing Team
- 3) Operations Team

Note: For every team member jenkins login access will be provided.

Operations Team : Responsible to create/edit/delete jenkins jobs.

Development/Testing : Responsible only to run the jobs.

Note: By default, Every logged in user can perform all operations in Jenkins.

```
=====
Role Management in Jenkins
=====
```

=> We can provide access to users by using roles concept

=> In order to manage user permissions using role we can install below plugin

Plugin Name : Role-based Authorization Strategy

=> Create Role

=> Assign Permissions for the role

=> Create User
=> Add Users to the role

=====
How to take jenkins backup ?
=====

=> By using thin backup plugin we can take jenkins backup and we can restore also.

@@@ Reference video :: <https://www.youtube.com/watch?v=5Tb-AOUFuKQ&t=2s>

=====
Assignment : "GIT + Maven + Docker + kubernetes + Jenkins" Integration Project

@@ Project setup document : <https://github.com/ashokitschool/DevOps-Documents/blob/main/10-Jenkins-Docker-K8S.md>

@@ Project setup Video :: https://www.youtube.com/watch?v=1lBv1_iSLDw